



# GCP Data Engineering & AWS to GCP Migration

A comprehensive guide to modern data engineering on Google Cloud — covering BigQuery, Dataproc, Cloud Composer & Apache Airflow, Python & PySpark, and the dbt Data Build Tool — plus a practical, service-by-service guide to migrating from AWS (S3, Glue, Redshift) to GCP. Each chapter includes explanations, code examples, diagrams, cost guidance, and hands-on exercises.

## Table of Contents

### 1. Introduction to Data Engineering

Data engineering roles, batch vs streaming, lakes vs warehouses, the modern data stack

### 2. Google Cloud Data Platform

GCP data services, projects, IAM, regions, networking, cost model

### 3. BigQuery Fundamentals

Architecture, datasets and tables, SQL, partitioning, clustering, pricing

### 4. BigQuery Advanced & Optimization

Time travel, streaming, BigLake, external tables, BQML, performance and cost tuning

### 5. Dataproc & Managed Spark

Cluster types, ephemeral clusters, autoscaling, jobs, Spark & Hadoop on GCP

### 6. PySpark for Data Engineering

RDDs, DataFrames, Spark SQL, optimization, PySpark on Dataproc

## 7. Python for Data Engineering

Core Python, libraries, virtual environments, cloud client libraries, testing, packaging

---

## 8. Apache Airflow & Orchestration

Why orchestration, DAGs, operators, sensors, scheduling, idempotency

---

## 9. Cloud Composer in Practice

Managed Airflow on GCP, environments, DAG deployment, connections, monitoring

---

## 10. Building Reliable Data Pipelines

ETL/ELT design, patterns, error handling, retries, data quality, observability

---

## 11. dbt: The Data Build Tool

Transformations in the warehouse, models, materializations, tests, Jinja, seeds and snapshots

---

## 12. AWS Data Services & GCP Mapping

S3, Glue, Redshift, EMR, Kinesis and their GCP counterparts

---

## 13. AWS to GCP Migration in Practice

Strategies, S3 to GCS, Redshift to BigQuery, Glue/EMR to Dataproc, cutover, testing

---

## 14. Practice, Labs & Career Path

Hands-on labs, certifications, interview prep, career roadmap

---

## 15. References & Resources

Glossary, cheatsheet, key docs, migration checklists

---

# 1. Introduction to Data Engineering

## CHAPTER 1

## What is Data Engineering?

**Data engineering** is the discipline of designing, building, and maintaining the systems that collect, store, process, and make data available for analysis and machine learning. Data engineers build the pipelines and platforms that move data from source systems (databases, files, SaaS apps, streaming events) into storage and analytics systems where analysts, data scientists, and BI tools can use it.

In a typical organization, the data engineering layer sits between **source systems** (the operational systems that create data) and **consumers** (analysts, dashboards, ML models, applications). This book focuses on doing that work on **Google Cloud Platform (GCP)**, with a complete tour of AWS-to-GCP migration for teams coming from the AWS world.

## The Data Engineering Lifecycle

Modern data engineering is commonly described as a lifecycle with four stages:

| Stage          | What Happens                                                   | Example Tools (GCP)                                                     |
|----------------|----------------------------------------------------------------|-------------------------------------------------------------------------|
| Generation     | Source systems produce data: transactions, logs, events, files | Databases, SaaS APIs, IoT, application events                           |
| Ingestion      | Data is moved from sources into a landing area                 | Cloud Storage (GCS), Pub/Sub, Storage Transfer Service, connectors      |
| Transformation | Data is cleaned, structured, joined, and modeled for use       | Dataflow, Dataproc (Spark), Cloud Composer (Airflow), dbt, BigQuery SQL |

|         |                                                                |                                                  |
|---------|----------------------------------------------------------------|--------------------------------------------------|
| Serving | Data is made available for analytics, BI, ML, and applications | BigQuery, Bigtable, Cloud SQL, Looker, Vertex AI |
|---------|----------------------------------------------------------------|--------------------------------------------------|

Around this cycle sit the "undercurrents": **orchestration** (scheduling and dependency management), **data governance** (quality, lineage, cataloging, security), and **observability** (monitoring, alerting, SLAs). A pipeline that runs but is not orchestrated, monitored, or governed is not production-ready.

## Data Warehouses, Data Lakes, and Lakehouses

Storage and analytics architecture has evolved through three major patterns:

- **Data Warehouse:** structured, relational, highly optimized for SQL analytics. Data is cleaned and modeled before loading. (Example: BigQuery, Amazon Redshift.)
- **Data Lake:** stores raw data of any format (structured, semi-structured, unstructured) at low cost. Flexible but can become a "data swamp" without governance. (Example: Amazon S3, Google Cloud Storage.)
- **Lakehouse:** a modern hybrid — a data lake's cheap storage combined with warehouse-style SQL querying and management. Open table formats like Apache Iceberg, Delta Lake, and Hudi are the technical foundation. BigQuery (with BigLake and external tables) and Dataproc (with Iceberg/Delta) support lakehouse patterns.

| Aspect          | Warehouse               | Lake                    | Lakehouse                             |
|-----------------|-------------------------|-------------------------|---------------------------------------|
| Data format     | Structured/tabular      | Any                     | Any, with structured layers           |
| Schema          | Strict, defined at load | Schema-on-read          | Both                                  |
| Cost            | Higher per GB           | Lowest per GB           | Low storage + compute                 |
| Analytics speed | Fast SQL                | Slow without processing | Fast with engines like Spark/BigQuery |

|           |               |                                  |                             |
|-----------|---------------|----------------------------------|-----------------------------|
| Use cases | BI, reporting | Raw storage, ML, experimentation | BI + ML + streaming + files |
|-----------|---------------|----------------------------------|-----------------------------|

**Key Trend:** The boundary is blurring. BigQuery can query files directly in GCS (external tables), Dataproc can read and write BigQuery, and open formats like Iceberg let a single dataset serve both Spark and BigQuery. Modern teams design for this flexibility rather than picking one silo.

# 2. Google Cloud Data Platform

CHAPTER 2

## Google Cloud in One Picture

Google Cloud offers a broad portfolio of data services. Understanding how they fit together is the first step in architecting pipelines. The major data-related services:

| Category               | Services                                                 | Role                                                          |
|------------------------|----------------------------------------------------------|---------------------------------------------------------------|
| Object storage         | Cloud Storage (GCS)                                      | Cheap, durable storage for files and raw data; the lake layer |
| Analytics warehouse    | BigQuery                                                 | Serverless SQL data warehouse and analytics engine            |
| Distributed processing | Dataproc, Dataflow                                       | Managed Spark/Hadoop (Dataproc) and Beam pipelines (Dataflow) |
| Streaming              | Pub/Sub, Dataflow                                        | Event ingestion (Pub/Sub) and stream processing               |
| Databases              | Cloud SQL, Spanner, Bigtable, Firestore, Memorystore     | Operational data stores (relational, NoSQL)                   |
| Orchestration          | Cloud Composer                                           | Managed Apache Airflow                                        |
| Migration              | Storage Transfer Service, BigQuery Data Transfer Service | Move data into GCP                                            |

|            |                                               |                                             |
|------------|-----------------------------------------------|---------------------------------------------|
| Governance | Dataplex, Data Catalog, Data Loss Prevention  | Metadata, lineage, data quality, protection |
| BI & ML    | Looker, Looker Studio, BigQuery ML, Vertex AI | Consume and model data                      |

## Core Platform Concepts

Before building pipelines, you must understand GCP's organizational model:

- **Organization:** the root of your GCP hierarchy (your company).
- **Projects:** the fundamental unit of isolation and billing. Resources (buckets, datasets, clusters) live inside projects. Every API call is scoped to a project.
- **Folders:** organize projects hierarchically (e.g., by business unit, environment).
- **Regions and zones:** resources are created in a specific location. Data residency and latency depend on this choice.
- **IAM (Identity and Access Management):** who (principal) can do what (role) on which resource. Principles include least privilege and separation of duties.

```
# Setting project and region in gcloud
gcloud config set project my-data-project
gcloud config set compute/region us-central1
gcloud config set compute/zone us-central1-a
```

## Service Accounts and Application Default Credentials

Pipelines rarely authenticate as a human. They use **service accounts** — machine identities with IAM roles. Best practices:

- One service account per workload/pipeline, with least-privilege roles.
- Prefer short-lived credentials; rotate keys; use workload identity federation where possible.
- Never store keys in code or DAGs; use the metadata server (on GCE/GKE), Workload Identity (on GKE), or a secret manager for keys.

```
# Using Application Default Credentials in Python
from google.auth import default

credentials, project = default()
print("Using project:", project)
```

## Data Residency and Regions

Choosing regions matters for:

- **Compliance:** some data must stay in a country/region (GDPR, industry rules).
- **Performance:** keep compute near storage near consumers.
- **Availability:** multi-region storage options replicate across regions.
- **Cost:** data egress (leaving GCP or crossing regions) is billable — keep data movement inside a region when possible.

BigQuery has **regional** and **multi-regional** locations (e.g., `us`, `eu`). Cloud Storage supports regional, dual-region, and multi-region classes.

## Cost Model Essentials

GCP pricing is consumption-based. For data engineering the biggest cost drivers are:

- **BigQuery:** you pay for analysis (queries) and storage. The flat-rate (slot) option is for heavy, predictable usage; on-demand pricing charges per amount of data scanned per query. Designing for fewer scanned bytes is a core optimization skill.
- **Dataproc:** you pay for cluster VMs (per-vCPU/hour) plus a small Dataproc surcharge. Ephemeral clusters (create for the job, delete after) keep costs low.
- **Cloud Storage:** per GB stored per month, by storage class (Standard, Nearline, Coldline, Archive). Data access has additional charges.
- **Cloud Composer:** a monthly base cost plus worker/scheduler VMs and storage.
- **Network egress:** data leaving GCP is the hidden cost — minimize unnecessary egress.

**Cost mindset:** In the cloud, you pay for what you use, so engineering decisions (schema design, partition granularity, query patterns, cluster sizing) directly move the bill. A "cost-

aware" data engineer is more valuable than one who just writes working SQL.

# 3. BigQuery Fundamentals

---

## CHAPTER 3

### What is BigQuery?

**BigQuery** is Google Cloud's serverless, petabyte-scale data warehouse. "Serverless" means there are no servers to provision, patch, or scale: you write SQL, and Google's infrastructure runs it on a massively parallel compute fabric. You pay for what you query and store, not for capacity you reserve (unless you choose the flat-rate/slots model).

### Architecture: Why It's Fast

BigQuery's speed comes from three Google-internal systems:

- **Colossus:** the distributed file system that stores data in compressed, columnar format (Capacitor) on replicated storage. Storage is decoupled from compute.
- **Dremel:** the query execution engine that shards queries across thousands of workers, giving near-linear scaling. This is what makes petabyte scans possible in seconds.
- **Jupiter:** the high-speed network fabric connecting compute and storage — a single query can use over 1 Tbps of network bandwidth.

Two consequences shape everything you do: **columnar storage** (reading a few columns is much cheaper than reading whole rows) and **separation of storage and compute** (storing data is cheap; scanning it is what you pay for).

### Core Objects

| Object  | Description                           |
|---------|---------------------------------------|
| Project | Container for datasets; billing scope |

|         |                                                                                 |
|---------|---------------------------------------------------------------------------------|
| Dataset | Container for tables; access control boundary; has a region                     |
| Table   | Columnar data with a schema; can be native, external, or a view                 |
| Job     | An operation BigQuery performs (query, load, extract, copy); has an ID and logs |
| Slot    | Unit of query compute capacity (used for flat-rate pricing)                     |

```
# Fully-qualified table name  
project.dataset.table # e.g. myproj.analytics.orders
```

## Loading Data into BigQuery

Several loading patterns:

- **Batch load jobs** from GCS (Avro, Parquet, ORC, JSON, CSV) — fast, atomic, recommended.
- **Streaming inserts** (or the newer BigQuery Storage Write API) for near-real-time.
- **Data Transfer Service** for Google services (Google Ads, Ad Manager, etc.).
- **External tables** that query files in GCS directly without loading (see next chapter).

```
# Load from GCS using bq  
bq load --source_format=PARQUET \  
  myproj.analytics.orders \  
  gs://my-bucket/orders/*.parquet \  
  orders_schema.json
```

## BigQuery SQL Essentials

BigQuery uses GoogleSQL (ANSI-compliant). Key syntax points:

```
-- Basic query with SELECT  
SELECT  
  order_id,
```

```

customer_id,
total_amount
FROM
  `myproj.analytics.orders`
WHERE
  order_date >= '2026-01-01'
ORDER BY
  order_id DESC
LIMIT 100;

-- Aggregation with GROUP BY
SELECT
  DATE(order_ts) AS day,
  COUNT(*) AS orders,
  SUM(total_amount) AS revenue
FROM
  `myproj.analytics.orders`
WHERE
  order_ts >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
GROUP BY
  day
ORDER BY
  day;

```

- **Backticks** around fully-qualified table names containing project/dataset.
- Support for **structs**, **arrays**, **JSON**, geography, and ML types.
- Window functions, CTEs (WITH), and scripting (DECLARE, FOR, IF) for multi-step logic.

## Partitioning: The #1 Cost and Performance Lever

**Partitioning** physically divides a table into segments by a column (usually a date or timestamp). Queries that filter on the partition column scan only the relevant segments instead of the whole table — dramatically cutting cost and improving speed.

- Partition by `DATE / TIMESTAMP` column, ingestion time ( `_PARTITIONTIME` ), or an integer range.
- Daily is most common; hourly and monthly are possible.
- Partition pruning happens automatically when the WHERE clause filters on the partition column.

```
CREATE TABLE myproj.analytics.orders (  
  order_id INT64,  
  order_ts TIMESTAMP,  
  total_amount NUMERIC  
)  
PARTITION BY DATE(order_ts)  
OPTIONS (partition_expiration_days = 365);  
  
-- This query scans only the single day partition:  
SELECT COUNT(*) FROM myproj.analytics.orders  
WHERE DATE(order_ts) = '2026-08-01';
```

## Clustering

**Clustering** sorts data within partitions by one or more columns and stores them in blocks by sort key. It helps when queries filter on non-partition columns, and it helps join/group performance. Clustering adds no extra storage cost and can substantially reduce bytes scanned for selective filters on the cluster columns.

```
CREATE TABLE myproj.analytics.orders  
PARTITION BY DATE(order_ts)  
CLUSTER BY customer_id  
OPTIONS (partition_expiration_days = 365);
```

**Rule of thumb:** Partition by a high-cardinality time column used in most queries; cluster by columns used in filters and joins (customer, product, region). Keep cardinality in mind — clustering on a near-unique column like order\_id provides little benefit.

## 4. BigQuery Advanced & Optimization

---

### CHAPTER 4

## Time Travel and Snapshots

BigQuery keeps deleted or overwritten data accessible for a rolling window (7 days by default, configurable). You can query the table as it existed at a point in time:

```
-- Query a table as of 1 hour ago
SELECT * FROM myproj.analytics.orders
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR);
```

**Table snapshots** freeze a table at a point in time as a durable, cheap copy — ideal for disaster recovery, audit, and release management.

```
CREATE SNAPSHOT TABLE myproj.analytics.orders_snapshot_20260801
CLONE myproj.analytics.orders;
```

**Cloning** (via `CREATE TABLE ... CLONE`) creates a zero-copy reference that becomes a real table only when you write to it. This makes staging, dev environments, and "copy for analysis" nearly free until you modify them.

## Streaming and Real-Time Data

For near-real-time ingestion BigQuery offers two paths:

- **Legacy streaming inserts (`tabledata.insertAll`):** per-row inserts with 90-day retention of streaming buffer. Simpler but limited (no exactly-once, no automatic retry).
- **BigQuery Storage Write API:** the modern path — high throughput, exactly-once semantics, schema auto-detection, and default-on transactionality. Use it for production streaming.

Recommended production architecture: **Pub/Sub** → **Dataflow (or Dataproc streaming)** → **Storage Write API** → **BigQuery**. Dataflow provides exactly-once processing, autoscaling, and backpressure handling.

## External Tables and BigLake

BigQuery can query data that lives in **Cloud Storage** without loading it, using external tables. Modern capabilities:

- **BigLake tables:** external tables over GCS (or AWS S3 / Azure ADLS) with BigQuery's query engine, row-level security, and fine-grained access. Great for lakehouse patterns and when you can't (or shouldn't) copy data into BigQuery.
- **Open table formats:** BigQuery supports Apache Iceberg and Delta Lake tables, unifying lake and warehouse.
- **Query federation:** query data in Cloud SQL, Spanner, and other sources without loading.

```
# Create an external table over Parquet files in GCS
CREATE OR REPLACE EXTERNAL TABLE myproj.analytics.ext_events
WITH CONNECTION myproj.us.ext_conn
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://my-bucket/events/*.parquet']
);
```

## BigQuery ML: ML in SQL

**BigQuery ML (BQML)** lets you train and run ML models using SQL — no separate ML platform needed for many use cases:

```
-- Train a linear regression model predicting order amount
CREATE OR REPLACE MODEL myproj.analytics.revenue_model
OPTIONS (model_type = 'LINEAR_REG', input_label_cols = ['total_amount']) AS
SELECT
  customer_tenure_days,
  order_count,
```

```
total_amount
FROM myproj.analytics.customer_features;

-- Predict
SELECT * FROM ML.PREDICT(MODEL myproj.analytics.revenue_model,
  (SELECT customer_tenure_days, order_count, 0 AS total_amount
   FROM myproj.analytics.new_customers));
```

BQML supports linear/logistic regression, classification, clustering (k-means), time series (ARIMA), matrix factorization, and importing external models (TensorFlow, XGBoost). It is the fastest way to get an ML model from a warehouse table to production without leaving SQL.

## Performance Optimization: Slots and Query Shape

BigQuery executes queries in stages. Optimization focuses on reducing bytes scanned and avoiding skew:

- **Predicate pushdown:** filter as early as possible; the engine pushes filters down to storage.
- **Join order:** put the largest table first; the engine chooses join strategy automatically, but skewed keys cause stragglers.
- **Avoid cross joins;** use window functions where possible.
- **Use approximate aggregates** for huge distinct counts.
- **Cache:** BigQuery caches query results (for 24h) — identical queries are free if cached.
- **BI Engine:** in-memory acceleration for Looker/dashboard workloads.

**Optimization workflow:** (1) dry-run to see bytes scanned, (2) check the query plan's "most expensive stage", (3) eliminate unnecessary columns and filters, (4) verify partitioning/clustering match the filter pattern, (5) consider a materialized view for repeated aggregates.

# 5. Dataproc & Managed Spark

## CHAPTER 5

### What is Dataproc?

**Dataproc** is Google Cloud's managed service for **Apache Spark, Hadoop, Flink, Presto/Trino**, and related open-source data processing frameworks. It gives you clusters of virtual machines pre-installed with these engines, managed for you — you create a cluster, submit jobs, and let it scale or shut down. You pay for the underlying VMs plus a small managed service fee.

Dataproc is the natural home on GCP for workloads that already run on open-source tools (especially those migrating from AWS EMR, as we'll cover in Chapter 13).

### When to Use Dataproc vs Dataflow vs BigQuery

| Tool     | Best For                                                                      | Model                            |
|----------|-------------------------------------------------------------------------------|----------------------------------|
| BigQuery | SQL analytics, warehouse workloads, BI                                        | Serverless SQL                   |
| Dataflow | Streaming and batch pipelines with Beam (portable code, unified model)        | Serverless, autoscaling          |
| Dataproc | Open-source Spark/Hadoop/Flink workloads, ML with PySpark, migrating EMR jobs | Managed VMs, full engine control |

Choose Dataproc when you have existing Spark code, need specific engine versions, use ML libraries that require Spark, or want control over cluster configuration. Choose Dataflow for new unified batch+stream pipelines. Choose BigQuery when SQL is enough.

### Cluster Types

- **Standard cluster:** a long-running cluster of master + worker VMs, used for many jobs. Simple but can waste money when idle.
- **Single node cluster:** one VM runs both master and worker roles — ideal for development and small jobs.
- **Ephemeral (transient) clusters:** created per job, deleted when the job finishes. The cost-efficient default for scheduled batch jobs.
- **Autoscaling clusters:** workers scale up and down based on utilization. Requires careful sizing of secondary worker pools.

```
# Create an ephemeral cluster and run a job in one step
gcloud dataproc jobs submit pyspark \
  --cluster=my-cluster \
  --region=us-central1 \
  gs://my-bucket/etl.py

# Cluster itself is created via:
gcloud dataproc clusters create my-cluster \
  --region=us-central1 \
  --master-machine-type n2-standard-4 \
  --worker-machine-type n2-standard-8 \
  --num-workers 4
```

## Job Submission

Dataproc supports several job types: PySpark, Spark (Scala/Java), Spark SQL, Hadoop MapReduce, Hive, Pig, Flink, Trino/Presto, and shell. Jobs can be submitted:

- Via the **gcloud CLI** or console (for manual runs).
- Via the **REST API** from Cloud Composer/Airflow (the production pattern).
- As **Workflow Templates** — predefined job sequences with dependencies, parameters, and orchestration, which Composer/Airflow can trigger.

```
# Submit Spark SQL job
gcloud dataproc jobs submit spark-sql \
  --cluster=my-cluster --region=us-central1 \
  --file=gs://my-bucket/queries.sql
```

## Components and Customization

- **Optional components:** Jupyter, Zeppelin, Docker, Anaconda, Presto, Flink, Zookeeper, etc. — enabled at cluster creation.
- **Initialization actions:** scripts that run on cluster VMs at startup (install packages, mount drives, configure clients).
- **Custom images:** pre-built images with your libraries for faster, reproducible startup.

```
# Enable Jupyter notebook component
gcloud dataproc clusters create ml-cluster \
  --region=us-central1 \
  --optional-components=JUPYTER \
  --enable-component-gateway
```

## Integrating with BigQuery and GCS

Dataproc natively reads and writes Google services:

- **GCS:** the `gs://` filesystem is available out of the box — Spark jobs read/write files directly (no copying to HDFS).
- **BigQuery:** the Spark BigQuery connector ( `com.google.cloud.spark.bigquery` ) lets Spark DataFrames read and write BigQuery tables with the Storage Read/Write API. This is how you bridge "Spark processing" and "warehouse analytics".
- **Pub/Sub:** the Spark Pub/Sub connector enables streaming ingestion.

```
# PySpark: read from BigQuery
df = spark.read.format("bigquery").option("table", "myproj.analytics.orders").load()
# write back
df.write.format("bigquery").option("table", "myproj.analytics.orders_processed").mode("append")
```

## Cost Management

- Prefer **ephemeral clusters** for scheduled jobs — pay only while jobs run.

- Use **preemptible (spot) workers** for fault-tolerant, cost-sensitive workloads — they can be reclaimed, so design jobs to tolerate it.
- Right-size VMs: monitor CPU/memory with Cloud Monitoring and adjust.
- Use **secondary worker pools** (scaled-down spot nodes without HDFS) for pure compute.
- Turn off idle standard clusters via scheduled autoscaling.

**Cost math example:** An ephemeral cluster with 1 master (n2-standard-4) + 4 workers (n2-standard-8), running 2 hours/day: roughly 5 vCPUs master + 32 worker vCPUs × 2h/day × ~30 days — a small fraction of leaving a 24x7 cluster running. Ephemeral clusters commonly cut Dataproc bills by 60-90%.

# 6. PySpark for Data Engineering

---

## CHAPTER 6

### What is PySpark?

**PySpark** is the Python API for **Apache Spark**, the open-source unified engine for large-scale data processing. PySpark lets you write Python code that runs distributed across a cluster, processing data that is too big for a single machine. On GCP, PySpark runs on **Dataproc** (or Dataproc Serverless).

### The Spark Execution Model

Understanding Spark's model explains most performance behavior:

- **Driver:** runs your main program, builds the execution plan, schedules tasks.
- **Executors:** worker processes on cluster nodes that run tasks and store cached data.
- **Tasks:** units of work, one per partition; the parallelization unit.
- **Stage:** a group of tasks that can run without shuffling data.
- **Shuffle:** redistribution of data across partitions (e.g., for group-by or join) — the expensive operation to minimize.
- **Lazy evaluation:** transformations build a plan; nothing runs until an action (collect, count, write) forces execution. This lets Spark optimize the plan.

### DataFrames: The Main API

For data engineering, you use the **DataFrame** API — a distributed table with named columns, similar in feel to pandas/SQL but distributed.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
```

```

spark = SparkSession.builder.appName("sales_etl").getOrCreate()

# Read Parquet from GCS
df = spark.read.parquet("gs://my-bucket/sales/*.parquet")

# Transform
result = (
    df.filter(F.col("amount") > 0)
      .withColumn("year_month", F.date_format(F.col("sale_ts"), "yyyy-MM"))
      .groupBy("year_month", "region")
      .agg(
        F.sum("amount").alias("revenue"),
        F.countDistinct("customer_id").alias("customers")
      )
      .orderBy("year_month")
)

# Show (action) and write (action)
result.show(10)
result.write.mode("overwrite").parquet("gs://my-bucket/agg/monthly_sales/")

```

## RDDs and the Low-Level API

Before DataFrames, the API was **RDDs** (Resilient Distributed Datasets). DataFrames are the modern default (optimized via the Catalyst optimizer and Tungsten execution). RDDs are still useful for low-level control and custom operations, but prefer DataFrames/SQL for data engineering:

```

# RDD example (rarely needed, but good to recognize)
rdd = spark.sparkContext.textFile("gs://my-bucket/logs/*.log")
counts = rdd.flatMap(lambda line: line.split(" ")) \
              .map(lambda word: (word, 1)) \
              .reduceByKey(lambda a, b: a + b)
for w, c in counts.take(10):
    print(w, c)

```

## Spark SQL: The Engine's SQL

Spark SQL lets you register DataFrames as tables and query them with SQL — the sweet spot when logic is clearer in SQL:

```
df.createOrReplaceTempView("sales")

result = spark.sql("""
    SELECT date_format(sale_ts, 'yyyy-MM') AS year_month,
           region,
           sum(amount) AS revenue
    FROM sales
    WHERE amount > 0
    GROUP BY year_month, region
    ORDER BY year_month
""")
result.write.mode("overwrite").saveAsTable("analytics.monthly_sales")
```

DataFrames, Spark SQL, and PySpark code all compile to the same optimized physical plans — pick whichever expresses the logic best.

## Optimization Fundamentals

- **Minimize shuffles:** group/join by aligned partition keys, use `coalesce` instead of `repartition` when shrinking.
- **Choose file formats well:** Parquet (columnar, compressed) over CSV/JSON for most workloads.
- **Bucketing:** pre-bucket tables on join keys so joins become shuffle-free.
- **Broadcast small tables:** `broadcast(df_small)` avoids shuffling a big table in joins.
- **Filter early:** push predicates down; Spark pushes filters into file readers (Parquet) automatically.
- **Avoid per-row Python UDFs on hot paths** — use native SQL functions; use Pandas UDFs ( `pandas_udf` ) for vectorized operations.
- **Manage partition counts:** too few partitions under-utilize the cluster; too many create scheduling overhead. ~100-200MB per partition is a good rule of thumb.

**Join strategy example:** joining a 1 billion-row fact table with a 1 million-row dimension — broadcasting the dimension avoids a full shuffle. Joining two huge tables requires

aligning bucketing or letting Spark choose sort-merge join. Understanding this trade-off is the #1 Spark performance skill.

# 7. Python for Data Engineering

---

## CHAPTER 7

### Why Python?

Python is the de facto language of data engineering. It is the scripting layer for orchestration (Airflow DAGs are Python), distributed processing (PySpark), ingestion (client libraries), transformation (dbt custom tests, Airbyte), and ML. A strong Python foundation multiplies everything else in this book.

### Python Fundamentals You Actually Use

- **Data structures:** dicts, lists, tuples, sets; comprehension and generator patterns for transforming data.
- **Functions and typing:** well-typed signatures with type hints; dataclasses for row-like structures.
- **Iterators and lazy processing:** reading files line-by-line without loading all into memory.
- **Exceptions and logging:** structured try/except, the `logging` module, never bare `except`.
- **Context managers:** `with open(...)` for safe resource handling.
- **Virtual environments and dependency management:** venv/pip and lock files (uv, pip-tools).

```
from dataclasses import dataclass
from typing import Iterator
import logging

logger = logging.getLogger(__name__)

@dataclass
class OrderRow:
```

```

order_id: int
customer_id: int
amount: float

def parse_orders(path: str) -> Iterator[OrderRow]:
    with open(path) as f:
        for line in f:
            parts = line.strip().split(",")
            try:
                yield OrderRow(int(parts[0]), int(parts[1]), float(parts[2]))
            except (ValueError, IndexError) as e:
                logger.warning("Skipping bad line %r: %s", line, e)

# Usage
for row in parse_orders("orders.csv"):
    print(row.order_id, row.amount)

```

## The Essential Libraries

| Library                     | Use                                                                         |
|-----------------------------|-----------------------------------------------------------------------------|
| pandas                      | In-memory tabular data for small-to-medium data, exploration, local testing |
| numpy                       | Numerical arrays, math on vectors                                           |
| requests / httpx            | Calling REST APIs (SaaS sources, internal services)                         |
| google-cloud-bigquery       | BigQuery client (loads, queries, jobs)                                      |
| google-cloud-storage        | GCS client (objects, transfer)                                              |
| google-cloud-pubsub         | Pub/Sub publish/subscribe                                                   |
| google-cloud-secret-manager | Store and fetch secrets                                                     |

|                       |                                              |
|-----------------------|----------------------------------------------|
| SQLAlchemy / psycopg2 | Relational DB access                         |
| PyYAML / Jinja2       | Config parsing and templating (dbt, Airflow) |
| pytest                | Testing                                      |

## Cloud Client Libraries and Application Default Credentials

The Google Cloud Python libraries authenticate via **Application Default Credentials (ADC)**: on GCE/GKE/Airflow workers they use the metadata server; locally they use `gcloud auth application-default login`; in CI they use service account keys or Workload Identity Federation.

```
from google.cloud import storage, bigquery

# GCS
client = storage.Client(project="my-data-project")
bucket = client.bucket("my-bucket")
blob = bucket.blob("raw/orders_20260801.csv")
blob.upload_from_filename("orders_20260801.csv")

# BigQuery: run a query
bq = bigquery.Client()
query_job = bq.query("SELECT COUNT(*) AS n FROM myproj.analytics.orders")
print(query_job.result().to_dataframe())
```

## Writing Robust Scripts

- **Entry points with argparse:** every job takes parameters (date, environment, project) — no hardcoded values.
- **Retries:** wrap flaky API calls in retry logic with exponential backoff (tenacity or manual).
- **Idempotency:** the script should produce the same result whether run once or re-run (overwrite, upsert, or partitioned writes).
- **Logging:** structured, queryable logs with enough context to debug (job id, parameters, timings).

- **Exit codes:** fail loudly with non-zero exit codes so the orchestrator (Airflow) can detect and retry.

```
import argparse
import logging
from google.cloud import bigquery

logger = logging.getLogger("load_orders")
logging.basicConfig(level=logging.INFO)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source", required=True)
    parser.add_argument("--destination", required=True)
    parser.add_argument("--date", required=True)
    args = parser.parse_args()

    logger.info("Starting load: %s -> %s (%s)", args.source, args.destination, args.date)
    client = bigquery.Client()
    # ... do the work ...
    logger.info("Completed")

if __name__ == "__main__":
    main()
```

## Packaging for Dataproc and Composer

Distributed jobs should be packaged and versioned:

- Build a **wheel** of your code ( `pip wheel .` ) and reference it in the Spark submit.
- Ship a **requirements.txt** or container image with pinned versions for reproducibility.
- Use **Dataproc custom images** or **initialization actions** to install your package on clusters.
- For Cloud Composer, install custom Python packages into the environment or use an `airflow worker` with dependencies preloaded.

```
# Build and submit a packaged job to Dataproc
pip wheel --no-deps -w ./wheels .
gcloud dataproc jobs submit pyspark \
  --cluster=my-cluster --region=us-central1 \
  --py-files=gs://my-bucket/wheels/my_etl-1.0-py3-none-any.whl \
  gs://my-bucket/src/main.py -- --date=2026-08-01
```

## Testing and Code Quality

- Unit-test pure logic with pytest; mock cloud clients for I/O paths.
- Use **mypy** for type checking and **ruff**/black for style — cheap insurance on a shared codebase.
- Run tests in CI (GitHub Actions / Cloud Build) on every push.
- Test against real (small) cloud resources in a dev project before production.

## Security in Python Data Code

- Never commit secrets; use Secret Manager, environment variables, or Airflow connections.
- Sanitize inputs; avoid `eval` / `exec` on external strings (dangerous in DAG code).
- Use least-privilege service accounts; keep credentials out of logs (redact in logging).
- Scan dependencies (pip-audit, Dependabot) for vulnerabilities.

## Python + SQL: The Practical Combination

Most production data work is a mix: Python orchestrates, extracts, and glues; SQL transforms. dbt (Chapter 11) uses Python + Jinja to manage SQL models. PySpark gives SQL a distributed engine. Mastery means knowing when to reach for each — Python for control flow and I/O, SQL for set-based transformation.

**Practice Task:** Write a small Python package "orders\_loader" with: (1) a module that reads a CSV from GCS and validates rows (logging bad ones), (2) an argparse CLI that loads validated rows into a BigQuery table idempotently for a given date, (3) three pytest

tests with mocked GCS/BigQuery clients, (4) type hints checked by mypy. Package it as a wheel and submit it to Dataproc. Note: use a dev project or the free tier to avoid costs.

# 8. Apache Airflow & Orchestration

## CHAPTER 8

### Why Orchestration?

A data platform is many jobs: loads, transformations, quality checks, model refreshes. If jobs are run by cron and people, dependencies break silently, retries are ad-hoc, and nothing is auditable. **Workflow orchestration** solves this: it schedules jobs, manages dependencies between them, retries failures, and records what ran when.

**Apache Airflow** is the industry-standard open-source orchestrator. It turns workflows into Python code (**DAGs**) with dependencies, schedules, retries, and monitoring. On GCP, Airflow is offered as the managed service **Cloud Composer** (next chapter).

### Core Concepts

| Concept   | Description                                                                                                      |
|-----------|------------------------------------------------------------------------------------------------------------------|
| DAG       | Directed Acyclic Graph — the workflow definition in Python: tasks and their dependencies                         |
| Task      | A single unit of work in a DAG (e.g., run one SQL query)                                                         |
| Operator  | Class that defines what a task does (PythonOperator, BigQueryInsertJobOperator, DataprocSubmitJobOperator, etc.) |
| Sensor    | Special task that waits for a condition (e.g., a file to appear in GCS)                                          |
| Scheduler | Component that decides when tasks run based on schedule and dependencies                                         |
| Executor  | Runs tasks (local, Celery, Kubernetes executor)                                                                  |

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| DAG run       | An execution instance of a DAG for a specific schedule interval |
| Task instance | An execution of one task in a specific DAG run                  |

## A Minimal DAG

```
# dags/hello_gcp.py
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

def say_hello():
    print("Hello from Airflow")

with DAG(
    dag_id="hello_gcp",
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    default_args={"retries": 2, "retry_delay": timedelta(minutes=5)},
) as dag:
    t1 = PythonOperator(task_id="say_hello", python_callable=say_hello)

    t2 = BigQueryInsertJobOperator(
        task_id="run_query",
        configuration={
            "query": {
                "query": "SELECT COUNT(*) FROM myproj.analytics.orders",
                "useLegacySql": False,
            }
        },
    )

    t1 >> t2  # t2 depends on t1
```

## Schedule Semantics: Know Your Catchup

Airflow schedules by *data interval*: a run for day D executes after the interval closes (start at 00:00, runs at 00:05+). The `start_date` plus `schedule` defines which intervals run, and `catchup` determines whether missed past intervals are backfilled. The template variables `{{ ds }}` (data date) and `{{ execution_date }}` are the bread and butter for parameterizing jobs per day.

```
# Use the data interval date to build GCS paths
"gs://my-bucket/raw/orders/{{ ds }}/*.csv"
"{{ ds_nodash }}" # e.g. 20260801
```

**Common gotcha:** `catchup=True` on a newly created DAG with a distant `start_date` can fire hundreds of runs at once. Set `catchup=False` or backfill deliberately. Also remember Airflow's default timezone is UTC — set `timezone` and `start_date` explicitly to avoid off-by-one-day surprises.

# 9. Cloud Composer in Practice

---

## CHAPTER 9

## What is Cloud Composer?

**Cloud Composer** is Google Cloud's fully managed **Apache Airflow** service. You create an *environment* — which provisions the Airflow scheduler, webserver, workers, metadata database, and Redis cache — and you focus on writing DAGs. Google handles upgrades, scaling, and infrastructure, and it integrates natively with GCP (IAM, networking, Secret Manager, logging, monitoring).

## Environment Components

- **Webserver:** the Airflow UI — DAG list, graph views, logs, connections, variables, triggers.
- **Scheduler:** decides which tasks are ready to run; scheduler parallelism is your main tuning lever.
- **Workers:** execute tasks; can scale based on queue pressure.
- **Metadata database (Cloud SQL):** stores DAG and task state.
- **Redis:** the Celery message broker coordinating workers.
- **Cloud Storage bucket:** where DAGs, plugins, requirements, and data live.

```
# Create a Composer environment
gcloud composer environments create composer-prod \
  --location=us-central1 \
  --image-version=composer-2.22.0-airflow-2.7.3 \
  --node-count=3 \
  --environment-size=MEDIUM
```

## Deploying DAGs

Every environment has a GCS bucket; DAG files go into the `dags/` folder and the scheduler picks them up within a few minutes.

```
# Find the DAG bucket
gcloud composer environments describe composer-prod \
  --location=us-central1 \
  --format="value(config.dagGcsPrefix)"

# Sync DAGs from your repo
gcloud composer environments storage dags import \
  --environment=composer-prod --location=us-central1 \
  --source=./dags/
```

**CI/CD pattern:** commit DAGs to git; on merge, a pipeline validates with `airflow dags list` and deploys with `gsutil rsync`. Rollbacks are a redeploy of the previous commit. Keep DAG code in version control — it is application code, not configuration.

# 10. Building Reliable Data Pipelines

## CHAPTER 10

### Reliability Is the Job

Anyone can write a query that works once. Data engineering is the discipline of making it work *every day*, on schedule, correctly, and observably. This chapter turns the pieces from earlier chapters — BigQuery, Dataproc, Airflow/Composer, Python — into a systematic approach to building pipelines that don't break (and when they do, fail loudly and recover cleanly).

### Pipeline Design Patterns

| Pattern                     | Description                                      | When to Use                                                |
|-----------------------------|--------------------------------------------------|------------------------------------------------------------|
| Incremental load            | Load only new/changed data (by timestamp or ID)  | Default for high-volume tables; minimizes cost and runtime |
| Full load / snapshot        | Replace the whole table each run                 | Small tables, dimensions, reference data                   |
| Append-only with partitions | Add a new partition per run, never touch history | Event/fact data, immutable logs                            |
| Merge (upsert)              | Insert new rows and update changed ones          | Mutable operational data (orders, profiles)                |
| Landing → staging → curated | Layered lake: raw, cleaned, modeled              | Lakehouse architectures; keeps raw data for reprocessing   |

# The Medallion / Layered Lake Pattern

Data lands raw, is cleaned, then is modeled for consumption:

- **Bronze (raw/landing):** exactly as received, immutable, in GCS (Parquet/Avro/JSON).
- **Silver (cleaned/trusted):** validated, deduplicated, standardized schema, still granular.
- **Gold (curated/consumption):** aggregated, joined, business-ready for BI and ML.

This layering makes reprocessing safe (raw is never destroyed), makes lineage clear, and matches the dbt model philosophy (Chapter 11).

## Idempotency: The Golden Rule

An operation is **idempotent** if running it twice has the same effect as running it once.

Pipelines that fail partway and get re-run are inevitable, so every step must be idempotent:

- Write to date-partitioned destinations; write with `overwrite` per partition.
- Use merge/upsert instead of blind append for mutable data.
- Deduplicate before load (or dedupe at read time in the modeling layer).
- Avoid global "truncate table then load" patterns that break when the load fails.

```
# BigQuery: idempotent partition overwrite
bq query \
  --replace \
  --destination_table=myproj.analytics.orders_daily \
  --range_partitioning=order_date,0,1,1 \
  "SELECT ... FROM myproj.analytics.orders WHERE DATE(order_ts) = '{{ ds }}'"
```

## Error Handling and Retry Strategy

- Classify failures: **transient** (network blip, quota) → retry with backoff; **permanent** (bad schema, bad data) → fail loudly, alert, do not retry blindly.
- In Airflow: task-level `retries` + `retry_delay`, plus `trigger_rule` for downstream handling.
- Use dead-letter queues / quarantine tables for bad records instead of failing whole batches.

- Log the input path, job parameters, and error in every failure so debugging starts fast.

## Data Quality Checks

Automated checks are the difference between "pipeline ran" and "pipeline ran correctly."

Build a checks layer between stages:

- **Completeness:** row counts match expected ranges; no unexpected nulls.
- **Freshness:** data for the expected date/time actually arrived.
- **Uniqueness:** primary keys are unique.
- **Validity:** values within allowed domains (dates parse, amounts positive).
- **Consistency:** sums match upstream vs downstream (e.g., revenue in staging equals revenue in gold).

```
-- Example: freshness + count check as a SQL query that must return 0 rows
SELECT 'missing partition' AS check, 1 AS flag
WHERE NOT EXISTS (
  SELECT 1 FROM myproj.analytics.orders
  WHERE DATE(order_ts) = CURRENT_DATE - 1
);
```

dbt (next chapter) has built-in test types (unique, not\_null, accepted\_values, relationships) that formalize exactly these checks.

## Observability: Logging, Monitoring, Lineage

- **Structured logs:** JSON logs with job id, dates, counts, and durations; shipped to Cloud Logging.
- **Metrics:** record row counts, bytes, run durations to Cloud Monitoring; alert on anomalies (a 50% drop in rows is an incident).
- **Alerting:** Airflow failure alerts, freshness SLA alerts, quality-check failures.
- **Lineage:** Dataplex lineage shows which job produced which table — essential for incident response and audits.
- **Dashboards:** a "pipeline health" dashboard for the team (success/fail per DAG, MTTR for failures).

## Testing Pipelines

- **Unit:** pure functions, SQL snippets against small fixtures.
- **Integration:** run a DAG against a dev BigQuery/GCS with sample data.
- **Contract tests:** validate schemas between producer and consumer stages.
- **Replay/reconcile:** re-run a completed day in a sandbox and diff outputs — catches non-determinism and drift.

## Backfilling and Reprocessing

When logic changes, you must rebuild history. Design for it:

- Parameterize every job by date ( `--start_date --end_date` ).
- Keep raw (bronze) data so you can rebuild any layer.
- Use Airflow backfill ( `airflow dags backfill` or a trigger with start/end) for controlled reprocessing.
- Prefer partition-level rebuilds over full-table rebuilds.

## A Reference Reliable Pipeline

```
DAG: daily_orders_etl (schedule 06:00, data date {{ ds }})
```

```
├ wait_for_raw_files (GCS sensor)
```

```
├ load_raw (CSV → BigQuery bronze partition)
```

```
├ quality_raw (row counts > 0, no null order_id)
```

```
├ transform (dbt run / BigQuery SQL → silver + gold)
```

```
├ quality_gold (sums reconcile, uniqueness on PK)
```

```
└ notify (success summary to Slack; on failure → alert + page)
```

```
Retries: 2 with 5-min backoff on load/transform; no retry on quality fail.
```

```
Idempotency: partition overwrite everywhere; merge for mutable tables.
```

**Design review checklist:** Is every step idempotent? Can I re-run one day without touching others? Do I alert on freshness and counts, not just failures? Is raw data preserved for rebuilds? Do checks run between stages, not just at the end? If yes to all, the pipeline will survive contact with reality.

# 11. dbt: The Data Build Tool

## CHAPTER 11

### What is dbt?

**dbt (data build tool)** is the industry-standard tool for **transformations inside the warehouse** — the "T" of ELT. Instead of writing transformation code in Spark or scripts, dbt lets you define transformations as **SQL models**, version-controlled in git, with automatic dependency management, testing, and documentation. It connects to BigQuery (and Redshift, Snowflake, Postgres, etc.), compiles your models, and runs them in dependency order.

dbt is the tool that made ELT and "analytics engineering" a real discipline: transformations become reviewable, testable code that lives with the team.

### Core Concepts

| Concept         | Description                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| Model           | A SQL file ( <code>.sql</code> ) that defines a table or view — the core unit of transformation                  |
| Materialization | How a model is built: <code>view</code> , <code>table</code> , <code>incremental</code> , <code>ephemeral</code> |
| Seed            | A CSV loaded as a table (for small reference data)                                                               |
| Snapshot        | Type-2 history of slowly changing data                                                                           |
| Test            | Assertions about data quality (unique, not_null, relationships, custom)                                          |
| Source          | External tables dbt reads but doesn't own                                                                        |
| Macro / Jinja   | Templating for DRY SQL (loops, conditionals, reusable logic)                                                     |

Package

Reusable macros/models shared across projects (e.g., dbt-utils)

## Project Structure

```
my_project/
├─ dbt_project.yml      # project config, model paths, materializations
├─ profiles.yml         # connection profiles (targets: dev/prod)
├─ models/
│   ├─ staging/         # stg_* models: clean, rename, standardize
│   ├─ marts/           # business-ready models
│   └─ sources.yml      # declares source tables
├─ seeds/              # CSV reference data
├─ tests/              # custom generic tests
├─ snapshots/          # type-2 history models
└─ macros/             # reusable Jinja macros
```

## A First Model

```
-- models/staging/stg_orders.sql
SELECT
  order_id,
  customer_id,
  TIMESTAMP(order_ts) AS order_ts,
  SAFE_CAST(total_amount AS NUMERIC) AS total_amount
FROM {{ source('sales', 'orders') }}
WHERE order_ts IS NOT NULL
```

```
# models/marts/marketing/customer_revenue.sql
-- This model depends on stg_orders automatically
WITH orders AS (
  SELECT * FROM {{ ref('stg_orders') }}
)
SELECT
  customer_id,
  DATE(order_ts) AS day,
```

```
SUM(total_amount) AS revenue
FROM orders
GROUP BY customer_id, DATE(order_ts)
```

`{{ ref() }}` is the magic: dbt parses these to build the dependency graph and runs models in the correct order. `{{ source() }}` declares upstream data sources with tests and freshness checks.

## Materializations

| Materialization | Behavior                                             | Use When                                       |
|-----------------|------------------------------------------------------|------------------------------------------------|
| view            | A view; re-queries upstream each time                | Lightweight, cheap, staging                    |
| table           | Built as a table each run                            | Models used repeatedly or with heavy logic     |
| incremental     | Builds full first time, then inserts/merges new rows | Large append-only or slowly changing data      |
| ephemeral       | Inlined into dependents; not materialized            | Reusable logic that shouldn't exist as a table |

```
# dbt_project.yml – set default materialization
models:
  my_project:
    staging:
      +materialized: view
  marts:
    +materialized: table
```

## Incremental Models in BigQuery

```

{{
  config(
    materialized='incremental',
    partition_by={'field': 'order_date', 'data_type': 'date'},
    cluster_by=['customer_id']
  )
}}

SELECT
  order_id, customer_id, order_ts, total_amount,
  DATE(order_ts) AS order_date
FROM {{ source('sales', 'orders') }}
WHERE order_ts IS NOT NULL
{% if is_incremental() %}
  AND order_ts > (SELECT MAX(order_ts) FROM {{ this }})
{% endif %}

```

The `is_incremental()` guard adds the watermark predicate only on incremental runs, so the first run builds full history and later runs only process new data. Combined with BigQuery partitioning, incremental models are the standard for large tables.

## Tests: Data Quality as Code

```

# models/marts/marketing/_customer_revenue.yml
version: 2
models:
  - name: customer_revenue
    columns:
      - name: customer_id
        tests:
          - not_null
          - unique
      - name: revenue
        tests:
          - not_null
    tests:
      - dbt_utils.accepted_range:

```

```
column_name: revenue
min_value: 0
```

```
# Source freshness – alert if a source stops updating
sources:
  - name: sales
    tables:
      - name: orders
        freshness:
          warn_after: {count: 24, period: hour}
          error_after: {count: 48, period: hour}
        loaded_at_field: order_ts
```

`dbt test` runs every configured assertion. Custom generic tests and singular tests let you encode arbitrary business rules.

## Jinja and Macros

Jinja templates make models DRY — generate SQL for many tables, handle environments, and reuse logic:

```
-- macros/generate_schema_name.sql – standard multi-env schema handling
{% macro generate_schema_name(custom_schema_name, node) -%}
  {% if target.name == 'prod' and custom_schema_name %}
    {{ custom_schema_name | trim }}
  {% elif custom_schema_name %}
    {{ target.schema }}_{{ custom_schema_name | trim }}
  {% else %}
    {{ target.schema }}
  {% endif %}
{% endmacro %}
```

**Guidance:** start with plain SQL models and add Jinja only when repetition actually hurts. Over-templating makes models unreadable. The `dbt-utils` package covers most common needs (pivot, date spine, surrogate keys) without writing custom macros.

# 12. AWS Data Services & GCP Mapping

## CHAPTER 12

### Why Map AWS to GCP?

Many teams run data platforms on AWS and need to migrate to GCP — or simply want to understand both clouds. The concepts are nearly identical; the services and APIs differ. This chapter maps the AWS data services you are most likely to meet — **S3, Glue, Redshift, EMR, Kinesis** — to their GCP counterparts, with the differences that actually matter when migrating.

### The Service Mapping Table

| AWS Service            | What It Does                          | GCP Equivalent                                  |
|------------------------|---------------------------------------|-------------------------------------------------|
| S3                     | Object storage                        | Cloud Storage (GCS)                             |
| Redshift               | Data warehouse                        | BigQuery                                        |
| EMR                    | Managed Hadoop/Spark                  | Dataproc                                        |
| Glue                   | ETL service + Data Catalog + crawlers | Dataflow (ETL), Dataproc, Dataplex/Data Catalog |
| Kinesis (Data/Streams) | Streaming ingestion                   | Pub/Sub (+ Dataflow for processing)             |
| Data Pipeline          | Workflow orchestration                | Cloud Composer (Airflow)                        |
| Athena                 | Query files in S3 (serverless SQL)    | BigQuery external tables / BigLake              |

|                        |                       |                                            |
|------------------------|-----------------------|--------------------------------------------|
| DynamoDB               | NoSQL key-value       | Bigtable (or Firestore for simpler cases)  |
| RDS / Aurora           | Managed relational DB | Cloud SQL (MySQL/Postgres/SQL Server)      |
| Lake Formation         | Lake governance       | Dataplex                                   |
| Glue DataBrew          | Visual data prep      | Dataplex data quality / Dataflow templates |
| Managed Airflow (MWAA) | Managed Airflow       | Cloud Composer                             |

## S3 → Cloud Storage (GCS)

S3 and GCS are both object stores with buckets, objects, prefixes, versioning, and lifecycle policies. The differences that matter:

- **Naming and access:** `s3://bucket/key` → `gs://bucket/object` . AWS IAM roles → GCP IAM + service accounts; S3 ACLs/bucket policies → IAM and object-level ACLs; signed URLs → GCS signed URLs.
- **Consistency:** both are strong-consistency object stores now.
- **Storage classes:** Standard/IA/Glacier → Standard/Nearline/Coldline/Archive.
- **Tools:** `aws s3 cli` → `gsutil`; `boto3` → `google-cloud-storage`.

## Redshift → BigQuery

Redshift is a massively parallel (MPP) warehouse; BigQuery is a serverless warehouse. Both speak SQL, so this is the most mechanical migration — but the mental models differ:

| Aspect | Redshift | BigQuery |
|--------|----------|----------|
|--------|----------|----------|

|                |                                             |                                                   |
|----------------|---------------------------------------------|---------------------------------------------------|
| Infrastructure | Clusters you size and scale (nodes, slices) | Serverless; pay per query + storage               |
| Scaling        | Add/remove nodes; distribution keys matter  | Automatic; optimize partitions/clustering instead |
| Sort/dist keys | DISTKEY + SORTKEY                           | Partitioning + clustering                         |
| SQL dialect    | Postgres-flavored                           | GoogleSQL (ANSI + extensions)                     |
| Compression    | Compression encodings per column            | Automatic (columnar, Capacitor)                   |
| Streaming      | COPY + STL system tables                    | Streaming API / Storage Write API                 |

Unload from Redshift (UNLOAD to S3 as Parquet/CSV) then load into BigQuery via `bq load` or the Transfer Service. Rewrite DISTKEY/SORTKEY-heavy SQL as partitioning/clustering and let BigQuery's optimizer handle the rest.

## EMR → Dataproc

EMR and Dataproc both run Spark/Hadoop clusters on cloud VMs. Migration is mostly the same open-source stack in a new management plane:

- **Cluster config:** EMR instance types/spot → Dataproc machine types + preemptible workers.
- **Filesystems:** `s3://` → `gs://` (Dataproc reads GCS natively).
- **Jobs:** `spark-submit` stays the same; `gcloud/API` replace `aws CLI`.
- **Bootstrap:** EMR bootstrap actions → Dataproc initialization actions.
- **Metastore:** EMR Hive metastore → Dataproc Metastore (managed Hive metastore) or external.
- **Best practice:** Dataproc ephemeral clusters (create per job, delete after) — a pattern EMR supports but teams rarely adopt as aggressively.

## Glue → GCP

Glue is AWS's ETL service with three distinct parts, each mapping differently:

- **Glue Jobs (Spark ETL):** a managed Spark environment. On GCP, run the same Spark ETL on **Dataproc** (more control) or **Dataflow** (if you rewrite to Beam). Glue's PySpark code ports to Dataproc PySpark with small changes (filesystems, session bootstrap).
- **Glue Data Catalog + Crawlers:** schema discovery and metadata. GCP equivalent is **Data Catalog** plus **Dataplex** for governance; schema discovery can be scripted or done via Dataproc/BigQuery auto-detect.
- **Glue Data Quality:** automated quality rules → Dataplex data quality / dbt tests.

## Kinesis → Pub/Sub

- **Kinesis Data Streams** (retained, replayable stream) → **Pub/Sub** (with message retention and replay) — both are pull/push event buses.
- **Kinesis Data Firehose** (load streams into S3/Redshift) → **Dataflow** streaming to GCS/BigQuery, or Pub/Sub subscriptions writing to BigQuery.
- **Kinesis Analytics** (SQL on streams) → Dataflow SQL / Dataflow with Beam SQL.

## Athena → BigQuery External / BigLake

Athena queries files in S3 directly. On GCP, the equivalent is querying files in GCS via BigQuery **external tables** and **BigLake** — same idea (schema-on-read, serverless SQL over object storage), stronger warehouse integration.

## MWAA (Managed Airflow) → Cloud Composer

Both are managed Airflow. If you use MWAA today, your DAGs port to Composer with minimal change — update provider packages (boto3/aws operators → google provider operators), adjust connections/secrets handling, and redeploy the same DAGs to the Composer bucket.

## Conceptual Differences That Trip People Up

- **IAM vs IAM:** AWS policies on resources (JSON) vs GCP roles bound to principals. Mindset shift, not a hard change.
- **Regions/AZs vs regions/zones:** similar; data residency rules differ per cloud.
- **Billing:** AWS and GCP both consumption-based, but warehouse pricing differs sharply (Redshift = reserved cluster compute; BigQuery = per-byte-scanned analysis + storage). Migration changes the cost shape and requires query redesign to stay cheap.
- **Networking:** VPC + security groups → VPC + firewall rules; PrivateLink → Private Service Connect.

## Dual-Cloud Patterns

Not every migration is "rip out AWS." Many teams run dual-cloud: data lives in both (via replication), or GCP is used for analytics while AWS remains the operational source. Tools like BigQuery Omni and GCS's S3/Azure interop make reading AWS S3 data from BigQuery possible without copying. Decide your pattern — full migration, parallel run, or dual-cloud — before touching infrastructure.

**Practice Task:** For a company running Redshift + Glue (Spark) + S3 on AWS, produce a service-by-service mapping table to GCP. For each service, note: (1) the GCP counterpart, (2) what code/config must change, (3) what can be reused as-is, and (4) one specific migration risk (data residency, SQL dialect, cost shape, IAM model). Share the table and identify which migration is hardest and why.

# 13. AWS to GCP Migration in Practice

## CHAPTER 13

### Migration Strategies

There are four levels of migration, in increasing order of change:

| Strategy       | Approach                                                               | Effort  | Benefit                             |
|----------------|------------------------------------------------------------------------|---------|-------------------------------------|
| Lift and shift | Same architecture, same scripts, GCP services swapped one-for-one      | Low     | Fast; limited cloud-native gains    |
| Re-platform    | Use managed services (S3→GCS, Redshift→BigQuery) with code translation | Medium  | Operational simplicity, right tools |
| Re-architect   | Redesign pipelines for serverless/ELT (Composer + dbt + BigQuery)      | High    | Full value: cost, scale, speed      |
| Rebuild        | New data platform from scratch on GCP                                  | Highest | Cleanest outcome; most risk         |

Most successful migrations do **re-platform first, re-architect gradually**. Move data and SQL now; optimize architecture in waves.

### Phase 0: Discovery and Inventory

- Inventory all data sources: S3 buckets (sizes, classes), Redshift tables (row counts, sizes, dependencies), Glue jobs, EMR workloads, streaming streams, and downstream consumers.
- Map dependencies: which tables feed which dashboards, models, and reports.
- Assess data volume and growth; estimate GCP storage and query costs.

- Identify blockers: data residency requirements, encryption keys, third-party tool integrations, existing SLAs.

## Phase 1: Prepare the Target

- Create GCP project hierarchy with dev/staging/prod isolation.
- Design IAM: service accounts per workload, least privilege.
- Create GCS buckets matching your S3 layout (or better — a bronze/silver/gold design).
- Provision BigQuery datasets with partitioning/clustering plans.
- Set up Cloud Composer (or port MWAA DAGs) and the CI/CD pipeline for DAGs and dbt.

## Phase 2: Move the Data

### S3 → GCS

Options, in order of preference:

- **Storage Transfer Service** — the managed, resumable service for copying S3→GCS (handles huge volumes, scheduling, and progress tracking).
- **gsutil rsync** — simple CLI sync for smaller, one-time transfers.
- **BigQuery Omni / interop** — read S3 from BigQuery directly when full copy isn't needed yet.

```
# One-time S3→GCS sync
gsutil -m rsync -r s3://my-bucket/raw gs://my-bucket/raw
# (or set up Storage Transfer Service with a schedule for ongoing sync)
```

### Redshift → BigQuery

1. UNLOAD Redshift tables to S3 as Parquet (or CSV for non-Parquet-able types).
2. Transfer S3 → GCS.
3. Load into BigQuery via `bq load` or the Data Transfer Service.
4. Migrate views/models as BigQuery SQL; replace DISTKEY/SORTKEY with partitioning/clustering.

```
-- Redshift: unload to Parquet
UNLOAD ('SELECT * FROM analytics.orders')
TO 's3://bucket/unload/orders'
IAM_ROLE 'arn:aws:iam:...:role/UnloadRole'
FORMAT AS PARQUET PARALLEL OFF;

-- BigQuery: load the Parquet
bq load --source_format=PARQUET \
  myproj.analytics.orders \
  gs://bucket/unload/orders/*.parquet
```

## Glue / EMR → Dataproc

- Port Glue PySpark scripts to Dataproc jobs: change `s3://` → `gs://`, replace Glue `DynamicFrame` idioms with standard Spark `DataFrames` where possible, and submit via Composer's `DataprocSubmitPySparkJobOperator`.
- Migrate Hive tables/metastore via Dataproc Metastore or re-create schemas in BigQuery.
- Use ephemeral clusters to control cost from day one.

## Phase 3: Migrate the Pipelines and Orchestration

- **MWAA → Cloud Composer:** port DAGs, swap AWS operators for Google providers, move connections to Secret Manager backend.
- **Data Pipeline / Step Functions → Composer:** rewrite the orchestration as Airflow DAGs.
- **Glue triggers / cron → Composer:** schedule loads and transformations centrally.
- **dbt:** switch the adapter from `dbt-redshift` to `dbt-bigquery`; most models run unchanged if SQL is portable (watch for Redshift-specific functions).

## Phase 4: Validation and Cutover

- **Parallel run:** run both platforms for a period and reconcile outputs (row counts, sums, freshness) before switching consumers.
- **Reconciliation queries:** compare key metrics table-by-table.

- **Cutover window:** freeze writes, sync the final delta, flip DNS/consumer connections, and monitor aggressively.
- **Rollback plan:** keep the AWS environment alive for 1-2 weeks (or per SLA) with a tested way to flip back.

## SQL Translation: Redshift → BigQuery Cheatsheet

| Redshift pattern        | BigQuery equivalent                         |
|-------------------------|---------------------------------------------|
| DISTKEY / SORTKEY       | Partitioning + clustering                   |
| SELECT TOP 100          | LIMIT 100                                   |
| DATEADD / DATEDIFF      | DATE_ADD / DATE_DIFF (or TIMESTAMP_ADD/SUB) |
| GETDATE()               | CURRENT_TIMESTAMP() / CURRENT_DATE          |
| LEFT/ RIGHT / SUBSTRING | SUBSTR / LEFT / RIGHT (same)                |
| NVL / ISNULL            | IFNULL / COALESCE                           |
| LISTAGG                 | STRING_AGG                                  |
| Regex                   | REGEXP_EXTRACT / REGEXP_CONTAINS            |
| GROUP BY 1,2 (ordinal)  | Not supported by default — use column names |

## Cost Control During Migration

- Run parallel platforms only as long as needed — double cost is temporary.
- Archive the old platform after cutover (don't keep paying for idle Redshift).
- Design BigQuery tables with partitioning/clustering to keep query costs low from day one.
- Monitor spend weekly during the transition.

**Top migration mistakes:** (1) copying Redshift's distribution-key design blindly into BigQuery; (2) forgetting data residency/compliance before moving; (3) migrating SQL that uses vendor-specific functions without translation; (4) no reconciliation step — "we think it's the same data"; (5) decommissioning AWS before the GCP pipeline has proven itself under real traffic.

# 14. Practice, Labs & Career Path

## CHAPTER 14

### Learning by Doing

Data engineering is an applied craft. This chapter gives you a realistic hands-on path using GCP's free tier and local tools, plus the certifications and career roadmap that turn skill into a job.

### Building a GCP Data Lab on the Cheap

| Component       | How to Get It (low-cost)                                                             |
|-----------------|--------------------------------------------------------------------------------------|
| GCP project     | Free trial + free tier; keep a dev-only project with budget alerts                   |
| BigQuery        | Free tier includes 1 TB/month of query and 10 GB storage — plenty to learn           |
| Cloud Storage   | 5 GB free; use Nearline/Coldline for practice data                                   |
| Dataproc        | Use single-node or ephemeral clusters, shut them down after; or run Spark locally    |
| Cloud Composer  | Minimum-size environment for short periods; or run Airflow locally via Docker (free) |
| dbt             | Local dbt-core + dbt-bigquery against the free BigQuery tier (free)                  |
| Public datasets | bigquery-public-data has massive free-to-query datasets                              |

**Cost safety:** Set a monthly budget + alert on your project before you start. Prefer ephemeral Dataproc and Composer environments that you delete after lab sessions. Most labs in this book run on the free tier.

# 15. References & Resources

## CHAPTER 15

### GCP Data Engineering Cheatsheet

| Concept             | Quick Reference                                                    |
|---------------------|--------------------------------------------------------------------|
| BigQuery            | Serverless warehouse; pay per byte scanned + storage               |
| Dataproc            | Managed Spark/Hadoop; ephemeral clusters for cost                  |
| Dataflow            | Managed Apache Beam (batch + streaming)                            |
| Cloud Composer      | Managed Apache Airflow; DAGs in a GCS bucket                       |
| Pub/Sub             | Managed message queue for streaming                                |
| Cloud Storage (GCS) | Object storage; classes Standard/Nearline/Coldline/Archive         |
| Partitioning        | BigQuery physical split by time column; prunes scans               |
| Clustering          | Sorts rows within partitions; speeds filters/joins                 |
| dbt                 | SQL transformation tool with tests, docs, lineage                  |
| ETL vs ELT          | Transform before load vs transform inside the warehouse            |
| Bronze/Silver/Gold  | Raw → cleaned → curated lakehouse layers                           |
| Idempotency         | Re-running yields the same result; the pipeline golden rule        |
| DAG                 | Airflow workflow: tasks + dependencies, scheduled by data interval |

|                          |                                                       |
|--------------------------|-------------------------------------------------------|
| catchup                  | Whether Airflow backfills missed historical intervals |
| Data interval / {{ ds }} | The date a run processes; always derive paths from it |

## AWS → GCP Mapping Cheatsheet

| AWS            | GCP                                | Key Change                                |
|----------------|------------------------------------|-------------------------------------------|
| S3             | Cloud Storage                      | s3:// → gs://; aws cli → gsutil           |
| Redshift       | BigQuery                           | DISTKEY/SORTKEY → partitioning/clustering |
| EMR            | Dataproc                           | Same Spark; ephemeral clusters            |
| Glue           | Dataflow / Dataproc + Data Catalog | ETL code ports to Dataproc PySpark        |
| Kinesis        | Pub/Sub + Dataflow                 | Streaming bus + processing                |
| MWAA           | Cloud Composer                     | Same Airflow; providers differ            |
| Athena         | BigLake / external tables          | Serverless SQL over object storage        |
| Lake Formation | Dataplex                           | Lake governance                           |

## SQL Translation Highlights

| Redshift  | BigQuery            |
|-----------|---------------------|
| GETDATE() | CURRENT_TIMESTAMP() |

|                      |                             |
|----------------------|-----------------------------|
| DATEADD('day', 1, d) | DATE_ADD(d, INTERVAL 1 DAY) |
| NVL(x, 0)            | IFNULL(x, 0)                |
| LISTAGG(x, ',')      | STRING_AGG(x, ',')          |
| GROUP BY 1           | GROUP BY (use names)        |
| LEFT/RIGHT/SUBSTRING | LEFT/RIGHT/SUBSTR           |

## Glossary

|                           |                                                              |
|---------------------------|--------------------------------------------------------------|
| <b>BigQuery ML (BQML)</b> | Training and running ML models with SQL                      |
| <b>BigLake</b>            | External tables with BigQuery engine and fine-grained access |
| <b>Capacitor</b>          | BigQuery's compressed columnar storage format                |
| <b>Catchup</b>            | Airflow's backfill of missed intervals                       |
| <b>Data interval</b>      | The period a scheduled DAG run processes                     |
| <b>Ephemeral cluster</b>  | Dataproc cluster created per job and deleted after           |
| <b>Egress</b>             | Network traffic leaving GCP — billable                       |
| <b>External table</b>     | BigQuery table that reads files in GCS without loading       |
| <b>Materialization</b>    | How dbt builds a model: view, table, incremental, ephemeral  |
| <b>Reconciliation</b>     | Verifying migrated data matches the source exactly           |
| <b>Slot</b>               | Unit of BigQuery query compute (flat-rate)                   |
| <b>Time travel</b>        | Querying a table as of a point in the past (7-day window)    |

## Key Resources

- **Google Cloud docs:** BigQuery, Dataproc, Cloud Composer, Storage, Pub/Sub, Dataflow, Dataplex.
- **Google Cloud skill-boost / free labs:** hands-on quests for Data Engineering.
- **dbt:** dbt Core docs, dbt Learn (free course), dbt-utils package.
- **Airflow:** Apache Airflow docs, official tutorial, google-cloud provider docs.
- **Public datasets:** bigquery-public-data, Kaggle datasets for practice.
- **Migration:** Google Cloud migration guides, Storage Transfer Service docs, Redshift-to-BigQuery migration guide.
- **Community:** r/dataengineering, dbt community Slack, Google Cloud community, Analytics Engineering meetups.

## Books and Deep Dives

- "Data Engineering on Google Cloud Platform" — Rohan Kumar (project-based GCP DE).
- "Google BigQuery: The Definitive Guide" — Lakshmanan & Tigani.
- "Fundamentals of Data Engineering" — Reis & Housley (design principles used throughout this book).
- "Analytics Engineering with SQL and dbt" — the dbt docs + community tutorials.
- "Learning Spark" / "Spark: The Definitive Guide" — for PySpark depth.

**Congratulations!** You have completed the GCP Data Engineering & AWS to GCP Migration Tutorial. You now understand the modern data stack on GCP — BigQuery, Dataproc/PySpark, Python, Airflow/Cloud Composer, and dbt — plus how to map and migrate an AWS platform (S3, Glue, Redshift) to GCP. Keep building labs, keep measuring cost, and keep practicing: data engineering rewards engineers who ship reliable, efficient pipelines.